Apuntes semana 2: lenguaje HCL

Oct 27, 2026 · @hector

Apuntes de los temas de teoría de la semana 2 del plan, con ejemplos basados en los labs en Azure.

Variables

Una variable se **declara** en `variables.tf` y se le **asigna** valor en otro lugar (por ejemplo `terraform.tfvars`). Se usa en el código como `var.nombre`.

Archivo	Función	¿Se sube a Git?
`variables.tf`	Declara: nombre, tipo, descripción y valor por defecto opcional	Sí
`terraform.tfvars`	Asigna el valor concreto para esta ejecución o ambiente	No, si tiene valores sensibles

Tipos

- Simples: `string`, `number`, `bool`
- Colecciones: `list(string)`, `set(string)`, `map(number)`
- Estructurales: `object({ nombre = string, tamano = number })` y `tuple([string, number])`
- `any`: Terraform deduce el tipo

Ejemplo completo

```
variable "location" {
  type        = string
  description = "Región de Azure"
  default     = "mexicocentral"

  validation {
    condition     = contains(["mexicocentral", "eastus"], var.location)
    error_message = "La región debe ser mexicocentral o eastus."
  }
}

variable "subscription_id" {
  type      = string
  sensitive = true
}
```

- `default`: si no existe, Terraform pide el valor al correr `plan`.
- `validation`: reglas propias. Se evalúan antes de crear nada, así que sí las detecta `validate` cuando el valor es conocido.
- `sensitive = true`: oculta el valor en la salida de `plan` y `apply`, pero **sigue guardándose en el state en texto plano**.

Regla práctica: `default` solo para valores estables y no sensibles; lo que cambia por ambiente o es privado va en `tfvars`.

Precedencia de variables

Si una variable recibe valor en varios lugares, gana el de mayor prioridad. De menor a mayor:

1. `default` en la declaración
2. Variables de entorno `TF_VAR_nombre` (por ejemplo `$env:TF_VAR_location = "eastus"`)
3. `terraform.tfvars`
4. `terraform.tfvars.json`
5. Archivos `*.auto.tfvars` y `*.auto.tfvars.json`, en orden alfabético
6. `-var` y `-var-file` en la línea de comandos, en el orden en que los escribas

Si una variable sin `default` no recibe valor en ninguno de estos, Terraform lo pide de forma interactiva.

Truco para recordarlo: entre más cerca del comando, más prioridad. Lo que escribes en la terminal le gana a todo.

Solo `terraform.tfvars` y `*.auto.tfvars` se cargan solos. Un archivo como `dev.tfvars` hay que pasarlo explícitamente: `terraform apply -var-file="dev.tfvars"`.